PromptABI: Static Verification for the Discrete Interface Layer of LLM Systems

PromptABI Contributors

June 4, 2026

Abstract

Large language model applications increasingly fail at a layer that is neither neural nor traditionally tested: the discrete interface layer made of chat templates, tokenizer metadata, special tokens, stop policies, structured output schemas, tool definitions, provider request envelopes, and framework truncation rules. PromptABI is a static and differential verifier for that layer. Its core claim is deliberately narrow and therefore practical: before loading model weights or running inference, a developer can prove that important prompt and protocol states are possible, impossible, ambiguous, or unsafe under the exact artifacts that will be used in production.

This paper presents the repository as it now stands and the research system it is designed to become. The implementation already contains a typed Python package, a deterministic command line workflow, a public embedding API, a typed artifact model, a stable diagnostic contract with text, JSON, and SARIF renderers locked by structured snapshots, position-aware source spans for configs and JSON artifacts, offline version-pinned artifact loading, a tokenizer abstraction and differential harness validated against real Hugging Face `tokenizers`, `tiktoken`, and SentencePiece libraries, deterministic finite automata, finite-state transducers with composition and projection, a finite-contract solver with compressed lazy product witnesses, incremental DFA minimization, indexed transducer composition/projection, solver timeouts, and constraint-sliced finite enumeration, byte-level tokenizer memoization for repeated normalization, a dependency-aware verification-session scheduler with deterministic parallel check execution and a per-session analysis cache, a typed plugin registry for artifact loaders, checks, provider adapters, grammar backends, template dialects, truncation policies, solver encodings, and diagnostic renderers, CLI-loadable module plugins, entry-point plugin discovery, and mode-aware plugin checks that preserve deterministic scheduling, focused tests, examples, a fixture corpus layout, a curated ten-family seed corpus, a deterministic corpus manifest pipeline, a Hugging Face `tokenizer_config.json` chat-template parser, a tokenizer/config drift checker that compares real tokenizer revisions for special-token ID changes, added-token changes, normalizer changes, chat-template hashes, BOS/EOS flags and IDs, and generation stop-policy deltas, deterministic PromptABI lockfiles that pin verified artifact hashes, upstream revisions, library versions, provider fixture versions, supported-fragment metadata, and expected diagnostic fingerprints for CI enforcement, a `promptabi init` workflow that writes verifiable starter contracts and local fixture stubs for OpenAI tools, Hugging Face local models, vLLM OpenAI-compatible servers, llama.cpp, LangChain RAG, LlamaIndex agents, and custom JSON Schema outputs, a `promptabi diff` workflow that compares two full model/provider/framework configurations and emits contract-breaking tokenizer, provider, framework, artifact-set, check-surface, and context-budget changes with structured witnesses and migration guidance, a stop-policy parser that normalizes OpenAI-compatible request snapshots, Hugging Face generation configs, llama.cpp/Ollama options, vLLM sampling params, LiteLLM passthrough params, and common framework wrapper kwargs into one typed artifact model with source spans, a stop-tokenizer analyzer that records byte/string/token alignments, multi-token stops, invalid stop-token IDs, normalization-sensitive stops, prefix/suffix collisions, and special or added token interactions against real tokenizer adapters, a fixture-backed stop differential harness that replays OpenAI-compatible, Hugging Face, llama.cpp-style, and vLLM-compatible stop traces through small CPU-only simulators, and a bounded stop-overreachability checker that constructs serialized witnesses for supported schema/tool string fields, nested JSON, markdown code fences, XML-like tool calls, and provider tool-call envelopes, including exact line/column firing points, parser states at truncation, and the malformed or prematurely accepted structures delivered to downstream parsers, a bounded symbolic executor for the supported Jinja chat-template fragment, a bounded role-boundary non-forgeability checker that detects unsanitized content and role fields capable of rendering control delimiters, special tokens, assistant prefixes, or tool-call sentinels, while recognizing explicit JSON, escape, and delimiter-safe wrapper filters, minimized role-forgery witnesses that include the malicious input field, rendered prompt excerpt, byte-level tokenization, and the exact forged boundary, a role-boundary model that decomposes symbolic renders into system, user, assistant, tool, developer, function, and provider-specific structural regions, a concrete renderer differentially checked against Hugging Face `apply_chat_template` behavior across the seed corpus, a delimiter- collision regression suite covering ChatML, Llama header tokens, Mistral instruction tags, XML-ish tool tags, markdown

role fences, and fine-tune hash headers, a real-world bug corpus that reduces public llama.cpp, vLLM, and Hugging Face Transformers reports into synthetic offline fixtures for Phi-style role delimiter forgery, ChatML special-token injection, Qwen/Gemma-style tool argument truncation, streaming stop interruption, array-of-object tool-argument leakage, multi-function XML boundary confusion, and XML tool-call parser-boundary stops, a grammar ingestion layer that normalizes JSON Schema, regex, EBNF, Outlines, xgrammar, llguidance, and PromptABI grammars into typed rules, terminals, source spans, supported-fragment metadata, and explicit abstentions, plus a JSON Schema path that normalizes objects, arrays, enums, consts, required fields, numeric and string constraints, type and composition unions, bounded local references, recursion-limit abstentions, and additional-property semantics, then compiles the supported subset into grammar IR, parser states, DFA witnesses, source maps, and real `jsonschema` validator round-trips, replays a CPU-only grammar differential corpus with hand-labeled Outlines, xgrammar, llguidance, lm-format-enforcer, Guidance, Instructor, and Pydantic-derived semantics through the CLI as agreement, mismatch, or abstention diagnostics, a structured-output parser compatibility checker that replays bounded grammar/schema witnesses and declared runtime samples against JSON, JSON Schema, XML-ish tool-call, markdown-fence, and custom-delimiter parser models while separating parser-broader from grammar-broader disagreements and explicitly marking the result as heuristic evidence rather than language equivalence, a labeled structured-schema corpus with open-source-agent reductions, anonymized production-style contracts, synthetic stress cases, tool-definition bundles, per-entry PromptABI configs, deterministic hashes, and manifest generation, a tool/function schema ingestion layer that normalizes OpenAI tools, Anthropic tools, LangChain tools, Pydantic model schemas, TypeScript-style functions, MCP tools, and provider tool-call envelopes into source-mapped typed function schemas, plus a bounded tool-call serialization checker that compares recorded provider, template, stop-policy, and parser contracts for tool names, JSON-object versus JSON-string arguments, escaping guarantees, tool-call IDs, parallel calls, streaming chunk assembly, and tool-call delimiters, a bounded provider migration checker that compares recorded OpenAI, Azure OpenAI, Anthropic, Gemini, Bedrock, Together, Groq, Ollama, llama.cpp server, vLLM OpenAI server, and LiteLLM contracts for request/response field loss, tool argument encoding, tool-call IDs, parallel-call support, streaming chunks, stop sequences, context limits, structured-output modes, error envelopes, and LiteLLM routing, and now takes the bounded tokenizer x grammar product to prove whether compiled witnesses survive real tokenizer encode-normalize-decode assumptions, plus a bounded tokenizer x grammar ambiguity check that validates JSON lexical variants and detects tokenizer-path conflicts, decoded-text conflicts, whitespace and Unicode byte aliases, and added-token aliases, a secret-free provider fixture pack corpus that records and replays request and response shapes, tool-call encodings, stop behavior, streaming deltas, error envelopes, context/output limits, and labeled edge cases for OpenAI, Anthropic, Gemini, Bedrock, vLLM OpenAI-compatible servers, and LiteLLM, with deterministic manifests, replay hashes, source-mapped diagnostics, and validation that rejects credential-like fields, a context-window budget model that combines named prompt segments, must-survive flags, declared or tokenizer-derived token counts, output reserves, tool-token reserves, generation-prompt tokens, and model-specific special-token overhead into bounded survival proofs and compact text/JSON/SARIF budget visualizations against real framework truncation policies, including minimized counterexamples when required regions can be dropped, a dedicated RAG chunking compatibility check that verifies tokenizer identity, chunk-boundary stability, citation survival, overlap accounting, metadata inflation, prompt-template overhead, and retrieval payload limits against the same packed prompt model, and a Z3-backed static-contract layer that derives finite SMT obligations from real templates, tokenizers, special-token maps, stop policies, tools, provider configs, truncation configs, prompt segments, and training/fine-tuning manifests, proving or refuting prompt-budget survival, stop/control-token exclusion, role-region non-forgeability, tool-schema required-parameter satisfiability, tool/provider compatibility, and supervised target-role alignment with concrete models, explicit proof classifications, or minimized unsat cores, a CPU-only benchmark suite, documentation, and contribution guidance. The CLI now also includes `promptabi explain`, which replays verification, deduplicates repeated logical diagnostics, selects one finding by fingerprint, rule, or index, and renders a tutorial explanation with source snippets, the checked formal property, proof-mode descriptions, witness steps, likely production symptoms, and concrete fixes in text or JSON. The remaining checkers fit into that surface rather than requiring a rewrite. We describe the transducer-oriented formalism, the artifact model, the diagnostic contract, the deployed benchmark lines, and the planned evaluation against real tokenizer, template, structured-output, provider, and budget failures. The diagnostic layer now records whether a finding is sound, complete, bounded, Z3-backed, heuristic, or an abstention, making each claim explicit about its proof obligations. The result is a path toward a tool that is useful in CI and credible as a systems paper: it catches failures that arise before any model logit is relevant.

1 Motivation

Modern LLM applications are built from small, fragile protocol objects. A chat template decides which bytes separate roles. A tokenizer decides whether those bytes are ordinary content or special tokens. A stop policy decides where output is truncated. A tool schema decides which JSON fragment is valid. A provider adapter decides whether a tool call appears as a message, a delta, a function object, or a model-specific sentinel. A framework budget policy decides which messages, retrieved chunks, and tool definitions survive when the prompt exceeds the context window.

These objects compose into a pipeline:

```
messages -> chat template -> byte/string prompt -> tokenizer -> token stream
        -> constrained decoder / stop logic / tool parser -> application parser
```

The failure modes are concrete. A user field can contain a string that renders as an assistant delimiter. A stop string can fire inside a valid JSON string, making the application parse a malformed prefix. A tokenizer update can change special-token IDs while tests still pass because they never inspect the token stream. A RAG prompt can preserve the latest user message while silently dropping the safety instruction or the tool definition that gives the message meaning. These are not model-behavior questions. They are contract questions.

PromptABI treats those contracts as first-class verification targets. It does not promise semantic prompt-injection resistance. It does not ask whether a model will follow an instruction. It asks whether the interface representation admits a structural state that the developer did not intend. That boundary makes the tool useful without GPUs, private model access, or nondeterministic generation.

2 Current Repository Artifact

The current repository milestone establishes the skeleton needed for serious verification work. It uses a Python `src/` layout, declares package metadata in `pyproject.toml`, exposes the `promptabi` console entrypoint, ships `py.typed`, and includes focused tests for the public API, artifact model, diagnostic contract, configuration loader, and CLI behavior. The implementation is intentionally small, but it is not a paper mockup: it loads real JSON configuration, discovers configs from subdirectories, accepts explicit artifact overrides, normalizes artifact paths, builds typed artifact bundles, records source spans for config declarations and parsed JSON artifact fields, materializes local and metadata-only remote artifacts, validates sha256 pins, adapts real tokenizer backends into stable encode/decode metadata, extracts Hugging Face chat-template structure from `tokenizer_config.json`, derives sanitizer-aware role-boundary regions over bounded rendered prompt patterns, emits minimized role-forgery witnesses with attacker input, rendered excerpt, token evidence centered on the forged marker, exact boundary offsets, and explicit sanitizer metadata, extracts automata, transducer, and solver witnesses for finite formal contracts, tracks check dependencies over artifact kinds, reuses cached tokenizer adapters, tokenizer/grammar analysis reports, budget reports, and static-contract query results, parallelizes independent embedded checks, serializes shared tokenizer and solver resources, fails closed on malformed JSON or UTF-8 artifacts, corrupt fixture archives, partial provider snapshots, invalid schemas, missing or incompatible tokenizer backends, and unsupported solver fragments by emitting source-mapped loader diagnostics or explicit abstentions, emits deterministic typed diagnostics in text, JSON, and SARIF with structured snapshot coverage, exposes an embedding API for custom checks, typed plugin registries, plugin artifact-loader hooks, plugin renderers, and typed results, ingests JSON Schema, regex, EBNF, Outlines, xgrammar, llguidance, and PromptABI grammar artifacts into typed rule and terminal summaries with source spans and abstention issues, normalizes supported JSON Schema objects, arrays, unions, references, constraints, and additional-property policies into a typed tree, compiles that tree into bounded grammar IR with parser states and DFA witnesses, and round-trips generated witnesses through the real `jsonschema` validator, checks grammar-backend differential fixtures against hand-authored accepted and rejected strings for Outlines, xgrammar, llguidance, lm-format-enforcer, Guidance, Instructor-style, and Pydantic-derived schemas, abstaining rather than overclaiming for fragments outside local replay semantics, compares structured-output grammar/schema artifacts with declared application parser models for JSON, JSON Schema, XML-ish tool calls, markdown fences, and custom delimiters using bounded witnesses and runtime samples, distinguishing parser-broader acceptance that can admit invalid application states from grammar-broader acceptance that can generate runtime-parse failures, checks tokenizer x grammar ambiguity by pairing schema-valid JSON lexical variants with real tokenizer encode/decode behavior to find conflicting token paths, decoded-text collapses, Unicode or whitespace byte aliases, and added-token aliases, checks recorded tool-call serialization contracts for provider/schema/parser agreement over names, argument encodings, escaping, IDs, parallel calls, streaming fragments, chat-template tool rendering, and stop delimiters, checks recorded provider migration contracts across all current first-party provider families for request, response, tool, stop, streaming, context, structured-output, error, and routing compatibility, validates provider fixture packs that capture secret-free request/response shapes, tool-call encodings, stop behavior, streaming deltas, error envelopes, and edge-case limits for offline replay, models context-window budgets from named prompt segments, must-survive flags, declared or real-tokenizer token counts, output reservations, reserved tool tokens, generation-prompt tokens, model-specific special-token overhead, bounded truncation-policy survival proofs with minimized failing segment sets, and machine-readable budget visualizations that expose spans, dropped fields, and survival guarantees, derives Z3-backed finite static contracts over cross-artifact obligations such as required-prompt-token budget survival, stop strings colliding with special or added control tokens, controlled prompt regions containing role/control boundary markers, tool schemas requiring parameters absent from their declared properties, tool-provider family mismatches, and training target roles outside the serving chat-template role universe, returns stable policy-controlled process exit codes, writes and enforces deterministic verification lockfiles that fail closed on artifact, config, provider-fixture, library-version, supported-fragment, or diagnostic-baseline drift, compares baseline and current PromptABI configs with `promptabi diff`, emitting text, JSON, or SARIF diagnostics for removed checks, missing artifacts, tokenizer/config drift, provider-contract regressions, framework truncation regressions, and top-level context-budget reductions, scaffolds common stacks with `promptabi init` and verifies every generated config through the same loader, provider replay, tool-schema, parser, budget, RAG, stop, and role-boundary checks used elsewhere in the test suite, expands a selected diagnostic with `promptabi explain` into a local debugging narrative that includes artifact snippets, the formal property, witness trace, likely production symptom, and fix suggestions while handling repeated logical findings deterministically, and runs CPU-only performance benchmarks over tokenizer, template, grammar, stop, SMT, budget, corpus, and cache paths.

The repository layout is designed to scale:

Path	Purpose
`src/promptabi`	Typed package, artifact model, formal automata, finite-state transducers, finite-contract core, offline loaders, CLI, configuration loading, diagnostic model, renderers, scheduler, analysis cache, and verification session.
`src/promptabi/plugins.py`	Typed extension registry for artifact loaders, dependency-aware checks, diagnostic renderers, and capability metadata covering provider adapters, grammar backends, template dialects, truncation policies, and solver encodings.
`tests`	Focused tests for package API and CLI determinism.
`examples/minimal`	A typed artifact config, messages file, tool schema, and answer schema used by tests and docs.
`examples/role-boundary`	Paired unsafe and sanitized ChatML-style `tokenizer_config.json` artifacts that exercise the real role-boundary non-forgeability CLI check.
`examples/stop-policies`	A vLLM sampling-parameter fixture that exercises real stop-policy artifact loading and normalization.
`fixtures`	Curated seed corpus, real-world bug reductions, grammar differential cases, labeled structured-schema/tool-calling fixtures, and secret-free provider fixture packs with provenance metadata, expected behaviors, reproducibility notes, runnable configs, and deterministic hashes.
`benchmarks`	CPU-only benchmark entrypoints for tokenizer analysis, symbolic template execution, grammar products, stop checks, SMT contracts, budget modeling, corpus verification, and cache reuse.
`docs`	MkDocs-ready documentation for architecture, concepts, and check families.

This milestone matters because later checkers must not accrete as one-off scripts. They need stable input objects, stable diagnostics, stable renderers, stable examples, and stable tests from the beginning. PromptABI now has that surface and a model that names each interface object explicitly.

3 Core Abstraction

The unifying abstraction is that LLM interface systems are compositions of finite-state transducers over byte, string, token, and structured alphabets, with partial parsers at the boundaries. A chat template maps structured messages to a string. A tokenizer maps strings to token streams and back, subject to special tokens, normalization, byte fallback, and added-token behavior. A constrained decoder or grammar describes a language of valid outputs. A stop policy cuts a prefix from that language. An application parser maps the resulting bytes back to an abstract syntax tree or rejects them.

Many high-value questions become language questions:

Property	**Question**
Reachability	Can a declared delimiter, stop sequence, or control token be produced under the selected tokenizer and grammar?
Over-reachability	Can the same sequence appear inside user-controlled content, a JSON string, a tool argument, or a markdown block?
Emptiness	Is the product of tokenizer assumptions and grammar constraints empty, so no valid output can be generated?
Non-forgeability	Can an attacker-controlled field render as role or provider structure after template expansion?
Must-survive	Does a required prompt segment remain after the framework truncation policy is applied?

The abstraction is not used to pretend arbitrary Python, arbitrary Jinja, and arbitrary provider behavior are finite-state. Instead, PromptABI classifies each check using a small contract taxonomy: *sound* means the diagnostic is not reported unless the modeled violation exists; *complete* means every violation inside the supported fragment is found; *bounded* means exactness is restricted to declared finite limits; *Z3-backed SMT* means the finite symbolic contract is lowered to Z3 when the optional solver is present; *heuristic* means the result is useful evidence rather than a proof; and *abstaining* means the checker deliberately declines unsupported cases. That taxonomy is crucial: it lets the tool be useful where it is precise and honest where it is not.

4 Artifact Model

PromptABI uses a typed artifact model because the same file can participate in multiple checks. A tokenizer config is needed for token-boundary analysis, special-token drift, chat-template parsing, and prompt-budget accounting. A tool definition is needed for role-boundary analysis, structured-output compatibility, provider migration checks, and budget survival. A diagnostic must therefore identify what artifact it refers to without coupling the renderer to a particular loader.

The public API now provides `ArtifactKind`, `ArtifactLocation`, `ArtifactProvenance`, `ArtifactBundle`, and typed artifacts for tokenizers, chat templates, special-token maps, stop policies, schemas, grammars, tool definitions, prompt segments, provider configs, framework truncation configs, and training/fine-tuning manifests. It also provides `ArtifactRef`, `SourceSpan`, `WitnessTrace`, `Diagnostic`, `VerificationConfig`, `VerificationSession`, and `VerificationResult`. These types establish the compatibility contract:

- artifacts have stable kinds, names, local paths or URIs, provenance, and optional versions, revisions, hashes, licenses, and sources;
- kind-specific fields capture stop strings, schema dialects, grammar types, tool names, prompt segment requirements, provider families, truncation strategies, and training message/target roles;
- spans are one-based, validated at construction time, and source-mapped from PromptABI configs plus tokenizer, schema, tool, provider, and fixture JSON artifacts;
- witnesses are explicit traces, not prose hidden in the message string;
- diagnostics have rule IDs, severities, artifacts, spans, witnesses, suggestions, and explicit verification modes;
- results serialize to deterministic dictionaries for JSON and snapshots.

The config loader exercises the model by accepting both legacy path strings and typed artifact objects, resolving local paths relative to the config file, attaching a declaration span to each artifact, and retaining URI-backed artifacts without subjecting them to local existence checks. This small behavior prevents a future class of flaky tests and confusing CLI output: diagnostics refer to the same canonical artifact no matter where the command is launched from, while missing-artifact and pinning findings point at the exact config entry that caused the run to inspect that artifact.

5 Formal Verification Core

PromptABI now includes the first executable formal core rather than only a roadmap for one. The core has two deliberately finite pieces. The first is a deterministic finite automata library over explicit alphabets. It constructs literal languages, finite-language tries, and prefix-closed literals; supports completion, complement, union, intersection, and difference; checks emptiness; extracts shortest witnesses with state-based breadth-first search; and serializes automata products deterministically. This is enough to express early reachability facts such as whether a stop string intersects a grammar prefix, whether a user-controlled field can equal a role delimiter, and whether removing forbidden control strings leaves any accepted user language.

The second piece is a finite contract solver. A contract declares variables over Boolean, enum, integer-range, and bounded-string domains, then composes named constraints with equality, disequality, integer comparisons, conjunction, disjunction, negation, implication, membership, string length, and substring containment. When `z3-solver` is available the adapter lowers those finite domains and constraints to Z3, including explicit domain restrictions for bounded strings and per-query timeout policies. When Z3 is absent, the same contract runs through deterministic finite-domain enumeration with constraint-variable slicing, partial assignment pruning, explicit assignment limits, and abstention rather than false proofs when a bound is reached. Both backends return a stable result object with `sat`, `unsat`, or `unknown` status, backend provenance, counterexample assignments for satisfiable bad states, explicit counterexample/proof/abstention conclusions, checked-assignment counts, and minimal unsat cores when no assignment exists. Z3 cores are deletion-minimized against the tracked named constraints while finite-domain restrictions remain background assumptions, so the proof object shown to a developer is not merely the first raw solver core.

The automata layer now optimizes the exact same properties rather than adding a separate approximate engine. It can find intersection witnesses lazily without materializing full products, compressing tokenizer-sized alphabets by equivalent left/right transition signatures at each product state. Eager union and difference remain on completed products so implicit sink-state semantics are preserved. Product construction minimizes completed operands incrementally before forming reachable products, and witnesses are reconstructed from predecessor maps instead of storing full paths in every queue entry. The transducer layer indexes right-hand transitions by input label during composition and reuses source transition indexes during projection closures, removing the nested scans that would otherwise dominate realistic render/tokenize/decode relations. The byte-level tokenizer also memoizes portable normalization results with a bounded cache, which removes repeated Unicode normalization work in corpus-wide stop, grammar, and budget checks without affecting encode/decode metadata.

The repository now uses that core as a real static-verification layer, not just as a toy solver. The `static-contracts` check derives independent cross-artifact obligations from loaded PromptABI artifacts. Required prompt segments and framework truncation configs become an integer SMT query asking whether required tokens can exceed the finite input budget. Stop policies, special-token maps, and tokenizer added-token metadata become a bounded string/enum query asking whether an application stop is also a model control token. Prompt segments, chat-template role universes, stop policies, special tokens, and tokenizer added-token metadata become a role-region non-forgeability query that asks whether controlled user, tool, function, or retrieval regions can contain a finite boundary marker. Source-mapped tool schema metadata becomes a required-parameter satisfiability query that proves every required argument is declared under the tool's properties, or returns a missing-parameter model. Tool definitions and provider configs become a provider-family compatibility query. Training manifests and chat-template role declarations become a target-role alignment query for supervised data pipelines. Satisfiable bad-state queries produce concrete model assignments and are reported as counterexample witnesses. Unsatisfiable queries produce proof diagnostics with the solver backend, minimized core, and a safety or incompatibility classification. Unsupported or insufficiently finite inputs abstain explicitly rather than widening into a claim the implementation cannot justify.

The core now adds the missing relational layer: finite-state transducers over explicit input and output alphabets. A transducer transition carries an input label, an output label, or epsilon on either side. This directly models render-like insertions, tokenizer-like recodings, decode-like normalization, and provider-envelope rewrites. The implementation accepts finite relations, literal string mappings, identity relations, exact composition over a shared middle alphabet, input and output projection back to deterministic automata, shortest input/output witnesses with path labels, and a conservative over-approximation that pairs the independent input and output projections when only a relation envelope is justified. Composition preserves the exact/over- approximate provenance flag so later diagnostics can distinguish proof from bounded approximation.

This split is important for the research claims. Automata should prove language and reachability properties; transducers should model interface mappings and their compositions; SMT should prove finite symbolic compatibil-

ity over discrete pipeline facts such as roles, escaping flags, bounded content, token budgets, schema preconditions, and stop-policy states. The fallback enumerator is not a replacement for richer solving, but it makes the supported fragment executable, testable, and privacy-preserving even in minimal CI environments.

The current tests exercise the core against real code rather than sketches. They prove that automata products extract a shortest stop-string witness, that difference excludes a role delimiter while preserving safe user strings, that transducer composition reconstructs a render-then-tokenize witness, that projections recover accepted input and output languages through epsilon labels, that projection-based over-approximation intentionally admits independent pairs, that a finite contract emits a concrete role-forgery counterexample, that contradictory role constraints produce a minimal unsat core, that Boolean compositions are recognized as unsatisfiable, that the installed Z3 backend returns the same kind of concrete assignment for a satisfiable finite contract, that raw Z3 cores are minimized deterministically before becoming diagnostics, and that CLI static-contract findings classify proof-of-safety and concrete counterexample outcomes in the witness trace. This is the first formal benchline on which role-boundary, stop-reachability, grammar-emptiness, and must-survive budget checkers can be built.

6 Version-Pinned Artifact Loading

Artifact identity is not enough for a verifier. A checker that proves a property about `tokenizer_config.json` must know which bytes, directory manifest, provider snapshot, or upstream revision it inspected. PromptABI therefore adds an offline loader layer that turns typed artifact references into deterministic `LoadedArtifact` summaries. The loader does not contact third-party services and does not require tokenizer, provider, or grammar libraries. It validates the reproducibility envelope first, so deeper semantic checkers inherit stable inputs.

The implemented loader covers the artifact sources that appear earliest in real LLM interface workflows:

Source	**Loaded summary**
Local files	Streamed sha256 digest, byte size, pin validation, JSON source-map extraction, and mismatch diagnostics.
Hugging Face refs	Offline repository ID, optional artifact path, revision extraction, and warnings for missing or movable revisions.
Tokenizer directories	Stable manifest over sorted relative file names, file sizes, and content hashes; requires tokenizer metadata.
GGUF stubs	Header-level GGUF magic, version, tensor count, metadata-key count, and streamed file hash without loading weights.
Provider snapshots	Deterministic JSON validation for provider name, captured request/response or streaming shape metadata, provider fixture pack sections, and optional stop-trace fixtures for offline differential replay, with parse-error line and column diagnostics.
Tool definitions	Source-mapped ingestion for OpenAI function tools, Anthropic `input_schema` tools, LangChain `args_schema` tools, Pydantic model JSON Schemas, TypeScript-style function arrays, MCP `inputSchema` tools, and provider tool-call response envelopes, recording tool names, argument encodings, closed parameter schemas, enum/constraint paths, IDs, streaming chunks, and ingestion issues.
Stop-policy configs	Deterministic extraction of stop strings, stop token IDs, EOS behavior, source family, ignored empty stops, and source paths from OpenAI-compatible, Hugging Face, llama.cpp/Ollama, vLLM, LiteLLM, and wrapper shapes.
Prompt segments, RAG chunks, and budgets	Source-mapped prompt segment files from explicit `segments` objects or OpenAI-style message arrays, merge behavior for config-declared must-survive regions, retrieval chunk metadata for document IDs, tokenizer names, source and packed boundaries, citations, overlaps, metadata costs, template wrapper overhead, payload limits, plus framework context-window artifacts with output, tool, generation-prompt, and special-token reservations.
Schema and grammar files	JSON Schema normalization and bounded compilation for objects, arrays, required fields, enums, consts, constraints, unions, bounded local references, recursion-limit abstentions, and additional properties; compiled metadata records grammar rules, parser states, DFA witness size, source spans, and real `jsonschema` round-trip acceptance, while tokenizer x grammar emptiness diagnostics prove whether bounded witnesses survive tokenizer encode/decode assumptions and tokenizer x grammar ambiguity diagnostics expose JSON lexical variants, Unicode escapes, whitespace, tokenizer normalization, and added-token paths that collapse or alias structured outputs. Ingestion for regex, EBNF, Outlines, xgrammar, llguidance, and PromptABI grammars records rules, terminals, references, source spans, and unsupported-fragment abstentions.
Fixture archives	Zip/tar member lists and archive-byte hashes without extraction or timestamp-dependent manifests.

Pinning is intentionally behavior-safe. A missing local file is an error because the verifier cannot inspect it. A malformed sha256 or mismatched sha256 is an error because the configured claim is contradicted. An otherwise readable but unpinned artifact is a warning by default, allowing existing projects to adopt strict reproducibility incrementally or fail on warnings in CI. In-memory API artifacts are treated as embedded objects rather than external supply-chain inputs.

This capability is more than bookkeeping. It is the first supply-chain and debuggability boundary for the research system: every future role-boundary, stop-reachability, grammar-emptiness, provider-migration, and budget-survival finding can cite the exact artifact reference, actual hash, revision strength, loader witness, and source span that made the claim reproducible.

7 Chat-Template Parsing and Symbolic Execution

PromptABI now parses Hugging Face `tokenizer_config.json` chat templates into a typed summary instead of treating them as opaque strings. The parser extracts the template source and its JSON source span, declared special tokens from standard tokenizer fields, message and tool field references, loops over `messages` and `tools`, role-condition literals, generation-prompt branches, filters, constants, whitespace-control usage, and unsupported Jinja constructs that require abstention by later symbolic checks.

This is intentionally a parser for verification inputs, not a general-purpose Jinja runtime. It recognizes the bounded fragment used by the seed corpus and records unsupported macros, imports, blocks, complex loops, and runtime globals such as `raise_exception`. The offline loader now recognizes chat-template artifacts backed by `tokenizer_config.json`, validates the template, attaches parsed metadata to the loaded artifact, preserves source spans, and keeps malformed or missing `chat_template` fields as deterministic loader errors. Focused tests run the parser across every curated seed-corpus template and targeted fixtures for tools, filters, whitespace controls, generation prompts, special-token declarations, and unsupported constructs. This creates the concrete input contract needed for bounded symbolic execution and role-boundary non-forgeability checks.

The next layer is now executable. PromptABI includes a position-preserving lexer and bounded symbolic executor for the supported Hugging Face Jinja fragment. It models loops over `messages` and `tools`, `if`/`elif`/`else` branches, simple `set` bindings, constants, special-token variables, message and tool fields, filters as symbolic wrappers, explicit whitespace controls, and Hugging Face's `trim_blocks`/`lstrip_blocks` structural whitespace behavior. The result is not a single rendered prompt but a finite set of symbolic paths with rendered patterns, path conditions, loop iteration counts, and explicit abstentions when unsupported constructs or path budgets are reached. Loader metadata records whether symbolic execution was precise and how many bounded paths were produced for a real artifact.

On top of those paths, PromptABI now builds a structural role-boundary model. For each symbolic rendered pattern it identifies message-indexed regions, records the conditions that prove a concrete role such as `system`, `user`, `assistant`, `tool`, `developer`, or `function`, keeps unconstrained role variables symbolic, preserves residual else-branch exclusions, and creates explicit generated-assistant-prefix regions. Region offsets point into the symbolic pattern, content expressions identify the controlled fields, and loader metadata summarizes role coverage and region counts for real `tokenizer_config.json` artifacts. This is the executable substrate for the non-forgeability checker rather than a prose-only design.

The first checker over that substrate is now implemented. It builds a global catalog of structural markers from declared special tokens, literal control fragments, generated assistant prefixes, role headers, and XML-like tool sentinels, then proves for each bounded symbolic path whether a raw user-controlled content field or dynamic role field can render one of those markers as structure. It explicitly flags direct `message['role']` interpolation as role-header forgeability and reports bounded, sound diagnostics with rendered excerpts. The checker is now sanitizer-aware at the exact expression level: JSON encoders such as `tojson`, HTML/XML escape filters, URL or base64 wrapper filters, and sanitizer-preserving `set` bindings are recorded in the role-boundary model and suppress only the findings for the protected expression. Raw interpolation remains forgeable, and Jinja's `safe` marker is deliberately not treated as escaping.

The regression suite now exercises the known delimiter-collision families against the executable checker rather than against string literals. ChatML cases verify `<|im_start|>`, `<|im_end|>`, assistant prefixes, and raw dynamic role headers. Llama-style cases cover start/end header IDs and end-of-turn tokens. Mistral-style cases cover `[INST]`, `[/INST]`, and EOS sentinels. XML-ish tool cases cover opening and closing tool-call tags, markdown cases cover role-bearing code fences, and custom fine-tune cases cover `### User:` / `### Assistant:` headers. The tests also assert that every expected finding carries a minimized malicious input, rendered excerpt, byte-level token evidence, and monotonic boundary offsets.

PromptABI also now has a concrete evaluator for that same supported fragment. It renders real message and tool inputs through the position-preserving lexer and bounded parser, evaluating role conditionals, generation-prompt branches, `messages` and `tools` loops, special-token variables, simple bindings, `loop.last`, and JSON filters without silently approximating unknown expressions. A reusable differential harness compares concrete renders with oracle strings collected from Hugging Face's real chat-template renderer. The deployed tests exercise every seed-corpus template with role/system variants, generation prompts, Unicode and sentinel-like content, structural whitespace, special-token insertion, empty tool lists, tool-role messages, and a focused tool loop using `tojson`. This closes the loop between the symbolic abstraction and the renderer developers actually run.

8 Tokenizer Abstraction Layer

Tokenizers are not a peripheral convenience in PromptABI; they are one of the central semantic boundaries. A role delimiter, stop string, JSON escape, or tool sentinel is only meaningful after normalization, added-token recognition, special-token handling, byte fallback, tokenization, and decoding are known. The repository now exposes a backend-neutral tokenizer interface that records token IDs, token surfaces, byte spans, special-token flags, added-token flags, normalization steps, decoded text, and round-trip metadata.

The implemented layer covers four practical families:

Backend	**Capability**
Byte-level	Deterministic byte alphabet with greedy added-token matching, portable Unicode normalization rules, UTF-8 byte spans, and exact decode reconstruction.
Hugging Face `tokenizers`	Local `tokenizer.json` and tokenizer-directory loading, real backend encode/decode, backend normalizer metadata, offsets converted to byte spans, and added-token detection.
`tiktoken`	Installed encoding adapters such as `cl100k_base`, special-token handling, token-byte metadata, and round-trip checks against the real library.
SentencePiece	Local `.model` loading, piece/id extraction, decode comparison, and best-effort byte spans where the piece surface can be matched.

This abstraction is deliberately thinner than a tokenizer reimplementation. Its job is to expose the observable behavior of real libraries in a stable shape that formal checks can consume. The implemented stop-tokenizer checker asks for byte spans, token IDs, token surfaces, decoded text, and special/added-token flags without knowing whether the source was ByteLevel BPE, SentencePiece, or `tiktoken`. A role-boundary checker can see whether a delimiter is an ordinary byte sequence, an added token, or a special token.

The current tests validate this layer against real code rather than mocks. A trained local Hugging Face ByteLevel BPE tokenizer is compared against `tokenizers.Tokenizer.encode` and `decode`. The `tiktoken` adapter is compared against `cl100k_base`, including special-token and byte-fallback behavior. A temporary SentencePiece model is trained and compared against `SentencePieceProcessor`. The byte-level adapter separately proves normalization, added-token, special-token, UTF-8 span, and round-trip behavior. These tests form the first differential benchline for the eventual tokenizer corpus.

8.1 Tokenizer Differential Harness

PromptABI now has a reusable differential harness rather than one-off adapter assertions. Each case records the input text, whether the backend should inject special tokens, whether decoding should skip them, and an oracle expectation collected from the real library. The harness then compares PromptABI's stable abstraction against that oracle across token IDs, token surfaces, decoded text, normalized text, normalization steps, special-token IDs, added-token IDs, required byte spans, and normalized round-trip behavior. Failures are returned as deterministic mismatch records with stable dictionaries suitable for future snapshot tests and corpus reports.

The harness already caught a realistic edge case during development: Hugging Face stores special tokens in its added-token decoder, but downstream structural checks need to distinguish provider/model control tokens from ordinary added tokens such as tool sentinels. The adapter now treats specialness and added-token status as separate flags, so a role-boundary checker can reason about `<s>` and `</s>` differently from an application-defined `<tool>` token. This is the sort of small tokenizer-library semantic difference that later formal checks must get right before their witnesses are credible.

8.2 Tokenizer and Config Drift

PromptABI now treats tokenizer upgrades as contract changes instead of informal release-note review. A tokenizer artifact can point at a local drift baseline via `metadata.drift_baseline_path`; the checker reads both baseline and current revisions from real `tokenizer_config.json`, `tokenizer.json`, `special_tokens_map.json`, and `generation_config.json` files. It compares special-token names, surfaces, and IDs; added-token IDs and specialness; normalizer signatures; chat-template hashes and lengths; BOS/EOS token text, IDs, and insertion flags; and stop strings or EOS stop-token IDs. Breaking changes emit deterministic CLI diagnostics with baseline/current values and revision witness steps; exact matches emit an informational proof that the baseline still

matches the current artifact. The focused tests build synthetic but real tokenizer directories and run the checker through `promptabi verify`, so the feature is exercised against the same loader, config, diagnostic, and renderer code used by users.

9 Diagnostic Contract

PromptABI's diagnostic contract is designed for CI, code scanning, embeddings, and regression snapshots. A useful finding is not merely a line of text. It needs a stable rule ID, a severity, a precise artifact reference with provenance, a source span when available, a witness that reproduces the failure, a deterministic fingerprint, a suggested remediation, and a declared verification mode. If two releases emit different diagnostics for the same artifact bundle, that difference should be intentional, reviewable, and testable.

The repository now renders the same diagnostic object in human text, deterministic JSON, and SARIF 2.1.0. This matters early: once a verifier has many rules, unstable output becomes a tax on CI, pull-request review, and paper reproducibility. PromptABI therefore fixes ordering, key names, optional-field behavior, and code-scanning fingerprints before high-volume checkers are introduced. A future role-forgery finding can reuse the same shape:

```
{
  "rule_id": "role-boundary-nonforgeability",
  "severity": "error",
  "fingerprint": "stable-sha256-prefix",
  "message": "user.content can render an assistant delimiter",
  "artifact": {
    "kind": "chat-template",
    "name": "tokenizer_config",
    "revision": "pinned-upstream-revision",
    "sha256": "artifact-hash"
  },
  "span": {"path": "tokenizer_config.json", "start_line": 12, "start_column": 3},
  "witness": {
    "summary": "malicious user content reaches assistant header",
    "steps": [
      {"action": "render template", "input": "user.content", "output": "..."},
      {"action": "tokenize prompt"},
      {"action": "locate forged boundary"}
    ]
  },
  "suggestions": ["Escape user content before inserting role delimiters."],
  "check_modes": ["bounded", "sound"]
}
```

The first implemented rule, `repository-skeleton`, is intentionally informational. It proves the machinery works without claiming a security or compatibility property that has not yet been implemented. That restraint is important for trust: checkers should only claim properties that the code has actually proved. The rule is therefore marked `heuristic`; loader and artifact-existence findings are marked `sound` and `complete` for the local filesystem and bytes they inspect; future automata and solver checkers can claim `bounded`, `z3-backed-smt`, or `abstaining` as their supported fragments require.

The SARIF renderer maps PromptABI severities to SARIF levels, emits a rule table, attaches artifact locations and source regions when available, stores the PromptABI fingerprint in `partialFingerprints`, and includes the same check-mode metadata as JSON and text output. This makes the first CLI workflow immediately compatible with GitHub code scanning while keeping the diagnostic source of truth in Python types rather than renderer-specific structures.

The source-region layer is executable rather than aspirational. PromptABI now builds a small recursive JSON source map over the exact text that is later decoded, converts offsets to one-based line and column spans, and keeps those spans outside generic metadata so JSON and SARIF remain serializable. Config artifact declarations carry spans into the typed artifact model; local JSON artifacts expose field-level spans for tokenizer configs, schemas, tool definitions, provider snapshots, and archived fixture members; malformed JSON loader errors carry the decoder's line and column. This gives early diagnostics the same property future formal witnesses need: a developer can jump directly to the config entry, `chat_template`, schema field, or tool declaration that made the finding possible.

The explanation renderer consumes the same diagnostic object rather than a parallel ad hoc format. It first collapses duplicate logical diagnostics by their stable structured payload, then lets users select by fingerprint, rule ID, or one-based index. A successful explanation includes the diagnostic message, fingerprint, artifact and source

span, a bounded local source excerpt when the artifact is available, the formal property associated with the rule, human descriptions of the proof modes, the witness trace, the likely production symptom, and the attached fix suggestions. This makes a code-scanning finding actionable without weakening CI determinism or sending prompts to a service.

10 CLI and Embedding API

The command line entrypoint is:

```
promptabi verify
promptabi verify --config examples/minimal/promptabi.json
promptabi verify --config examples/role-boundary/unsafe.promptabi.json
promptabi verify --config examples/role-boundary/safe.promptabi.json
promptabi explain --config examples/role-boundary/unsafe.promptabi.json --index 1
promptabi explain --config examples/minimal/promptabi.json --rule-id repository-skeleton --format json
promptabi init --stack openai-tools --output-dir .promptabi-demo
promptabi verify --artifact schema=schemas/answer.json --fail-on warning
promptabi verify --config examples/minimal/promptabi.json --format json
promptabi verify --config examples/minimal/promptabi.json --format sarif
promptabi verify --cache-dir .cache/promptabi --verbose
```

In the current milestone the workflow validates repository wiring, local existence, reproducible pins, loader compatibility, source-span reporting, and check-mode reporting. It discovers `promptabi.json` or `.promptabi.json` by walking upward from the current directory unless `-config` is explicit. It accepts repeated `-artifact NAME=PATH_OR_URI` overrides so CI can verify generated or matrix-selected artifacts without rewriting the repository config. It prepares a cache directory selected by `-cache-dir`, `PROMPTABI_CACHE_DIR`, `XDG_CACHE_HOME`, or a user cache fallback. It provides quiet and verbose text modes while keeping JSON and SARIF stable for machines.

`promptabi init` is deliberately not a static README snippet. Each stack template writes a `promptabi.json` plus minimal local artifacts that the real loader can ingest: tool schemas, provider replay packs, stop policies, Hugging Face `tokenizer_config.json` templates, prompt segments, truncation budgets, or JSON Schemas as appropriate. The focused init regression test generates every supported stack and verifies the resulting config through the public CLI, which makes the quickstart path an exercised benchline rather than a hand-maintained example.

The exit-code policy is configurable. The default fails only on error-level diagnostics. CI can instead fail on warnings, fail on any diagnostic, or never fail the process while still emitting machine-readable results. Malformed invocations and invalid configs return code 2, preserving the distinction between a broken workflow and a real artifact-contract finding.

The same behavior is available through the embedding API, which now covers the full first integration loop: loading a config, applying artifact overrides, loading artifacts through the deterministic offline loader, selecting built-in or custom checks, collecting diagnostics, returning a typed result, and rendering that result in the same formats as the CLI.

```
from promptabi import CheckContext, Diagnostic, DiagnosticSeverity, run_verification

def organization_policy(context: CheckContext):
    schema = context.artifact("schema")
    yield Diagnostic(
        rule_id="organization-schema-present",
        severity=DiagnosticSeverity.INFO,
        message=f"loaded {schema.artifact.name}",
        artifact=schema.artifact.to_ref(),
    )

result = run_verification(
    "promptabi.json",
    checks={"organization-schema-present": organization_policy},
    selected_checks=["organization-schema-present"],
)
if not result.ok:
    raise SystemExit(1)
```

The public helpers `create_session`, `load_artifacts`, `collect_diagnostics`, `run_verification`, and `render_result` deliberately mirror the CLI rather than bypassing it. The point is not that the current checker is deep. The point

is that all later checkers have an integration path. A LangChain plugin, a pre-commit hook, a GitHub Action, a notebook visualization, and a provider-fixture replay harness can all depend on the same session, loaded-artifact, diagnostic, and result objects. That prevents the common research-tool failure mode where each experiment has its own incompatible input format and output convention. The first workflow milestone also establishes practical CI affordances—discovery, overrides, caches, verbosity, deterministic rendering, explicit failure thresholds, and plugin-style check registration—before expensive formal checkers arrive.

The corpus maintenance workflow now has the same integration discipline:

```
promptabi corpus manifest
promptabi corpus manifest --output seed-corpus.manifest.json
```

This command validates the curated seed corpus without network calls, emits a stable manifest with per-entry metadata hashes, tokenizer-config hashes, fixture hashes, expected behaviors, license notes, upstream references, pinned fixture revisions, and reproducibility notes, and fails deterministically if a required family or sentinel contract is missing. That turns the corpus from a collection of files into an auditable experimental input.

11 Role-Boundary Non-Forgeability

The first major formal checker now starts from an executable role-boundary model and analyzes whether user, tool, function, or other attacker-controlled fields can render as structural prompt boundaries. In ChatML-like templates the relevant strings may be `<|im_start|>`, `<|im_end|>`, role names, or assistant prefixes. In Llama-style templates they may be header IDs and end-of-turn sentinels. In instruction-tag templates they may be `[INST]`, `[/INST]`, or custom fine-tune delimiters. In XML-like tool formats they may be tool-call start and end tags.

PromptABI already models the rendered prompt as regions: system, user, assistant, tool, developer, function, provider-specific symbolic roles, and generated assistant prefixes. The checker builds one global marker catalog for a chat template, split around placeholders so it never fabricates impossible concatenated markers. It then pairs every user-controlled content expression with every structural marker and separately checks raw dynamic role fields. Diagnostics identify the exact symbolic path and region, the controlled expression, the forged marker kind, a rendered excerpt showing where the forged boundary appears, and byte-level token evidence. The current implementation also records recognized sanitizers on role regions so JSON-encoded, escaped, URL/base64-wrapped, or sanitizer-preserving bound fields do not produce false-positive role-forgery reports.

This property is structural, not semantic. PromptABI can prove that a template allows user content to create an apparent assistant turn. It cannot prove that a model will obey that turn. That distinction makes the result more useful, not less, because the finding remains valid across model weights and sampling parameters.

The repository now exposes this distinction as an executable example rather than only a warning in prose. `examples/role-boundary/unsafe.promptabi.json` points at a ChatML-style template that directly interpolates `message['role']` and `message['content']`; the verifier fails with witnesses for raw `assistant`, `<|im_start|>`, and `<|im_end|>` markers. The paired `examples/role-boundary/safe.promptabi.json` uses the same control-token shape but JSON-encodes dynamic role and content fields, and the same check passes with no role-boundary diagnostics. Both artifacts are sha256-pinned and covered by focused CLI tests, so the documentation claim is exercised against real code.

12 Stop and Grammar Reachability

Stop policies often look simple: if the generated text contains a string, cut there. In structured-output systems this simplicity is dangerous. A stop string can be unreachable because the tokenizer or grammar cannot produce it. It can be ambiguous because multiple token paths or decoded byte strings correspond to the same visible text. It can overreach because the string appears inside a valid JSON string, tool argument, markdown fence, or XML-like field before the structured object is complete.

PromptABI now implements four concrete layers for this family: stop-policy parsing, tokenizer-backed stop alignment, fixture-backed stop differential replay, and bounded stop-overreachability. A dedicated parser accepts the shapes developers actually check into repositories or capture from serving stacks: OpenAI-compatible `stop` request fields, Hugging Face `generation_config` objects with `stop_strings` and EOS token IDs, llama.cpp/Ollama `reverse_prompt/antiprompt/ignore_eos` options, vLLM `sampling_params` with `stop_token_ids`, LiteLLM `litellm_params`, and nested framework kwargs such as `model_kwargs`, `generation_kwargs`, `parameters`, and `extra_body`. The loader normalizes those forms into a typed stop-policy artifact containing sorted unique stop sequences, stop token IDs, EOS behavior, source-family metadata, ignored empty stops, and JSON source spans. Malformed fields become deterministic loader errors rather than silent fallbacks.

The new tokenizer-backed layer analyzes every stop string under the selected tokenizer adapter and emits concrete byte/string/token alignment witnesses. It records UTF-8 bytes, token IDs, token surfaces, byte spans, normalized text, decoded text, round-trip metadata, multi-token segmentation, and special or added-token participation. It reports proper prefix and suffix collisions at the string, byte, and token-ID levels, normalization collisions where distinct configured stops collapse to the same tokenizer surface, and conservative unreachability findings for configured stop-token IDs that the chosen tokenizer cannot decode. The checker is intentionally careful not to claim that a string stop is unreachable merely because it does not round-trip through an encoder: many serving stacks match stop strings against decoded text streams. Those cases are reported as tokenizer-sensitive ambiguity instead, with heuristic modes.

The differential layer checks those static interpretations against recorded runtime traces without calling live providers. A provider-config fixture can store one or more `stop_trace` entries: generated text chunks, optional token IDs, the provider or framework family, and the expected stopped flag, matched stop, output text, finish reason, and whether the runtime retains the delimiter in the emitted output. PromptABI replays each trace through a small CPU-only simulator. OpenAI-compatible, vLLM, llama.cpp, Ollama, and LiteLLM-style families match raw substring stops on the cumulative decoded stream and exclude the stop delimiter from returned text; the Hugging Face stopping-criteria model records the common generated-text behavior where the stop string remains in the sequence that triggered stopping. Mismatches become deterministic diagnostics with replay witnesses, while agreement cases form an executable benchline. The repository ships both focused unit tests and a `examples/stop-policies` fixture that proves a vLLM-style trace agrees with the configured stop policy.

The overreachability layer separates structural delimiter collisions from supported content witnesses. Structural checks cover nested JSON delimiters, markdown code fences, XML-like tool-call tags, and OpenAI/Anthropic-style tool-call envelopes. Content checks are emitted only for supported JSON Schema and tool-parameter fragments where an unconstrained string field makes the stop text valid data, including bounded object and array-of-object parameter paths that occur in real tool schemas. All offsets are computed on the serialized wire text, including escaped characters, so the witness is not merely "the stop appears." It is a valid output prefix, the exact line and column where the stop fires, the parser state at truncation, and the malformed or prematurely accepted structure received by the application parser. PromptABI now analyzes the adjacent tokenizer x grammar emptiness product for bounded JSON Schema grammars, and should next combine stop policies with full grammars. A useful witness for grammar-level stop unreachability includes the stop sequence, its byte form, the tokenizer segmentation, and the grammar state that prevents production.

This check family is attractive because it is both practical and formal. It catches flaky production bugs, and it can be stated as a reachability property over a product of tokenizer, grammar, stop automaton, and parser states.

13 Structured Output and Grammar Emptiness

Structured-output libraries increasingly compile JSON Schema, regex fragments, EBNF-like grammars, Pydantic models, or provider-specific tool schemas into a decoder constraint. The application then parses the resulting bytes with a different parser. Bugs appear when those languages do not match. A schema may be valid under a Python validator but compile into an empty grammar under a specific backend. A grammar may accept strings the application parser rejects. A tokenizer may expose Unicode, escaping, whitespace, or added-token behavior that creates ambiguous structured outputs.

Tool schemas are now first-class inputs rather than opaque side files. PromptABI normalizes OpenAI `{type:function,functi` bundles, Anthropic `input_schema` tools, LangChain `args_schema` tools, Pydantic model JSON Schemas, TypeScript-style function arrays, MCP `inputSchema` tools, and provider response envelopes containing `tool_calls`. The result records the provider family, tool names, JSON-object versus JSON-string argument encodings, closed parameter schemas, required properties, enum and constraint paths, tool-call IDs, streaming chunk evidence, source spans, and ingestion issues such as open parameter objects. This gives later serialization, provider-migration, stop-overreach, and budget checks a typed schema substrate.

The next layer is now executable as `tool-serialization`. A recorded provider snapshot can declare the request tool names it accepts, the response tool-call names it emits, whether arguments are JSON objects or JSON-encoded strings, whether those strings are escaped, where tool-call IDs appear, whether parallel calls are enabled or observed, how streaming deltas fragment arguments, which parser names and encodings are accepted, and which delimiter belongs to the tool-call envelope. PromptABI compares that contract against the selected tool definition, chat template, stop policy, and parser declaration. It reports bounded diagnostics for name mismatches, argument-encoding mismatches, raw escaping risk, missing IDs, single-call parsers behind parallel providers, parsers that read each delta instead of assembling chunks, templates that do not render selected tools, and stop policies that omit the recorded tool-call delimiter. The fixture-backed test drives all of those findings through `promptabi verify`, with source spans into the provider JSON and witness steps for the compared fields.

The provider evidence is no longer an ad hoc collection of JSON snippets. PromptABI now ships a first-class provider fixture pack corpus. Each pack records the request surface, response envelope, tool-call name and argument paths, argument encoding, stop behavior, streaming delta assembly, error-code shape, and edge-case context/output limits for a representative provider family. The corpus loader validates those sections, materializes each pack as a loadable `provider-config` artifact, emits deterministic manifest hashes, and rejects credential-like keys or values such as authorization headers and API-key patterns. This gives provider-migration, tool-serialization, and future replay checks an offline input format that can be shared in bug reports and CI without storing secrets.

PromptABI now normalizes the supported JSON Schema subset into a typed tree that preserves objects, arrays, required fields, enum and const terminals, string and numeric constraints, type and composition unions, local reference expansion, bounded recursion abstentions, and additional-property semantics. It also compiles that tree into a bounded grammar IR with source maps, per-node parser states, a deterministic finite automaton over generated JSON witnesses, and round-trip checks against the real `jsonschema` Draft 2020-12 validator. The implemented `grammar-tokenizer-emptiness` check then pairs each local tokenizer artifact with each schema or grammar artifact. For the supported JSON Schema fragment it enumerates bounded DFA witnesses, encodes them with the real selected tokenizer adapter, decodes the resulting token IDs, and accepts only if the decoded text is still accepted by the grammar automaton. A satisfiable report records the JSON witness, token IDs, token surfaces, decoded text, grammar state path, and backend assumptions. An empty report shows the rejected candidate, such as a required `"OK"` literal rewritten by a lowercase-normalizing tokenizer into `"ok"`. Under the stated bounded assumptions, if that product is empty, then the constrained-decoding backend cannot produce a valid object through that tokenizer path.

The implemented `grammar-tokenizer-ambiguity` check adds the adjacent question: even when a schema is non-empty, can the interface represent it in more than one structurally relevant way? PromptABI generates bounded schema-valid JSON values, expands them into lexical variants such as compact and spaced encodings, Unicode spellings, escaped strings, and added-token-friendly text, validates the variants against the real `jsonschema` backend when available, and then compares equivalence classes over token IDs, decoded text, and parsed structured values. It reports token-path conflicts when distinct schema values collapse to the same token sequence, decoded-text conflicts when normalization erases a distinction, byte aliases when multiple JSON byte spellings parse to the same value under different token paths, and added-token aliases when a single added token and its byte spelling decode to the same grammar text. Each finding carries both grammar texts, both token paths, both structured values, decoded text, and the bounded assumptions under which the ambiguity was proven.

Equally important is abstention. Arbitrary schemas with recursive references, custom validators, remote ref-

erences, or backend-specific extensions should not be silently approximated as safe. The diagnostic should state exactly which fragment forced abstention and where that fragment appears.

14 Context-Window Budget Modeling

Token-budget failures are common because different layers count and truncate in different ways. A framework may drop old messages from the left. A RAG pipeline may trim chunks after adding metadata. A serving engine may reserve tokens for tools or generation prompts. A tokenizer mismatch may make a prompt fit in development but overflow in production. The result can be subtle: the user message survives, but the system instruction, output-format requirement, citation-bearing span, or tool schema disappears.

PromptABI now implements bounded must-survive verification over that modeling layer. A `prompt-segment` artifact names regions such as `system-policy`, `retrieval-context`, and `user-request`; marks regions as required or `must_survive`; carries declared `token_count` or `max_tokens` values; optionally includes content that can be counted through a real tokenizer adapter; and attaches per-segment overhead. The loader accepts explicit `{segments:[...]}` files and OpenAI-style message arrays, then merges file content with config-declared requirements so existing message fixtures become budget inputs without losing must-survive metadata.

A `framework-truncation-config` artifact contributes the maximum context window, output reservation, reserved tool tokens, generation-prompt tokens, model-specific special-token overhead, framework name, strategy label, model name, optional system/tool preservation flags, and role-priority drop hints. The `token-budget-model` check applies deterministic arithmetic:

```
input_budget =
  max_context
  - output_reserve
  - tool_reserve
  - generation_prompt_tokens
  - special_token_overhead
```

If a framework artifact and top-level config disagree about `max_context_tokens`, the artifact wins and the run emits an explicit conflict diagnostic. If required segments lack declared counts and no supported tokenizer can count their content, the checker abstains rather than inventing a tokenizer proxy. This preserves the repository's soundness boundary: arithmetic over declared or real-tokenizer counts is bounded and sound for the modeled inputs; estimates are not silently promoted into proof.

The check now normalizes framework-specific policies for LangChain, LlamaIndex, vLLM, Transformers, llama.cpp/Ollama, OpenAI-compatible servers, LiteLLM, common message-dropping strategies, and custom RAG pipelines. Left-truncating serving engines drop earliest prompt regions, conversation-memory frameworks drop oldest non-system messages by default, right and middle strategies are modeled explicitly, and RAG policies can prioritize retrieval-role loss before required instructions. Diagnostics report non-positive effective budgets, single segments larger than the input budget, required segment totals that cannot fit, total prompt overflow for optional content, optional segments dropped by the framework, and required segments that a concrete truncation policy can remove. For known counts and supported policies the checker also emits a `MustSurviveProof`: `proven` when every required region remains in the kept set, `violated` when truncation can remove one, and `abstained` when token counts or a custom policy fall outside the bounded model. Violations include a greedily minimized counterexample: the smallest ordered segment subset found by deletion that still makes the real policy drop a must-survive segment. The witness records the selected budget source, reserved-token arithmetic, known segment counts and their sources, unknown segments, kept segments, dropped segments, required-token sums, proof status, and counterexample token totals. The same diagnostic now carries a compact visualization row for every prompt segment: role, required/optional status, declared or tokenizer-derived count source, overhead contributions, cumulative token span, kept/dropped decision, truncation boundary, and survival label. This visualization is visible in human CLI text and is also emitted as the structured `token_budget_visualization` property in JSON and SARIF, so CI systems can annotate exactly which prompt fields disappear without scraping prose. The `examples/token-budget` workflow exercises this through the real CLI: a vLLM-style budget reserves output, tool, generation-prompt, and special-token overhead, then fails because required system and user segments exceed the modeled input budget and the selected truncation policy can remove required content.

The same prompt-segment path now carries RAG-specific chunk contracts rather than treating retrieved context as anonymous optional text. A retrieval chunk can declare its embedding or splitter tokenizer, document and chunk IDs, source character boundaries, packed prompt boundaries, expected and observed overlap, citation labels, citation-required flags, metadata token cost, prompt-template wrapper overhead, and a per-chunk retrieval payload limit. The `rag-chunking-compatibility` check reuses the budget simulator and emits bounded diagnostics when serving tokenizers disagree with chunking tokenizers, normalization or splitter drift changes chunk boundaries, citation-required chunks lose labels or are dropped by truncation, overlaps differ from the splitter contract, metadata

dominates useful content, wrapper text pushes chunks beyond their configured size, or the retrieved payload cannot fit its declared limit. The `examples/rag-chunking` workflow drives those cases through the real CLI and records deterministic JSON witnesses, making RAG failures concrete rather than a vague "context quality" concern.

15 Differential Validation

PromptABI's abstractions must be checked against the libraries developers actually use. The verifier should not implement a tokenizer, chat-template engine, or grammar compiler and assume equivalence. It should differentially test its abstraction against Hugging Face tokenizers, `apply_chat_template`, `tiktoken`, SentencePiece where practical, llama.cpp metadata, vLLM and OpenAI-compatible request fixtures, and structured-output backends such as Outlines, xgrammar, llguidance, lm-format-enforcer, Guidance, and Instructor.

The testing strategy has three tiers:

- unit and structured snapshot tests for public API and deterministic text, JSON, and SARIF rendering;
- tokenizer abstraction tests against real Hugging Face `tokenizers`, `tiktoken`, SentencePiece, and byte-level adapters;
- chat-template parser, symbolic-executor, concrete-renderer, role-region, and role-boundary non-forgeability tests across real `tokenizer_config.json` fixtures and known delimiter-collision regressions, including ChatML, Llama header tokens, Mistral instruction tags, XML-ish tool tags, markdown fences, fine-tune hash headers, generation-prompt extraction, dynamic role headers, content-control delimiter collisions, role assumptions, field references, special tokens, bounded message/tool loops, filters, whitespace controls, path budgets, empty tool lists, tool-role messages, and unsupported constructs, with concrete outputs differentially checked against Hugging Face's renderer;
- loader and parser tests for local files, Hugging Face refs, tokenizer directories, GGUF stubs, provider snapshots, tool-definition schemas spanning OpenAI, Anthropic, LangChain, Pydantic, TypeScript-style, MCP, and provider tool-call envelopes, tool-serialization contracts over names, arguments, escaping, IDs, parallel calls, streaming chunks, template tool rendering, and stop delimiters, stop-policy configs spanning OpenAI-compatible, Hugging Face, llama.cpp/Ollama, vLLM, LiteLLM, wrapper kwargs, fixture archives, and source-mapped JSON fields;
- stop-tokenizer, stop-differential, and stop-overreachability tests that exercise byte/string/token alignments, multi-token stops, invalid stop-token IDs, normalization collisions, prefix/suffix collisions, added/special-token interactions, OpenAI-compatible and Hugging Face delimiter-retention semantics, provider fixture replay, CLI mismatch diagnostics, serialized stop offsets, parser-state truncation witnesses, supported schema/tool string witnesses, structural delimiter collisions, real-world Qwen/Gemma/llama.cpp stop bug reductions, and a real Hugging Face `tokenizers` BPE fixture;
- token-budget and RAG chunking tests that exercise context-window reservation arithmetic, required-segment overflow over declared counts, prompt-segment file/message merging, config-versus-framework budget precedence, LangChain/vLLM/custom-RAG truncation simulation, optional retrieval dropping, required segment truncation, order preservation, proven must-survive cases, minimized counterexample extraction, text/JSON/SARIF budget visualizations with token spans and dropped-field properties, real CLI diagnostics for output, tool, generation-prompt, and special-token overhead, plus tokenizer mismatch, chunk-boundary drift, citation loss, overlap accounting, metadata inflation, prompt-template overhead, and retrieval payload truncation over repository fixtures;
- tokenizer/config drift and configuration-diff tests that build baseline and current tokenizer directories, provider snapshots, and framework truncation artifacts, then verify special-token ID drift, added-token changes, normalizer changes, chat-template hash changes, BOS insertion changes, generation stop-policy deltas, provider-contract regressions, removed checks, context-budget regressions, and loader-abstention behavior through real CLI diagnostics;
- fixture and corpus tests for minimized tokenizer/chat-template bundles, public real-bug provenance, expected behaviors, sentinel consistency, artifact materialization, deterministic manifest generation, labeled structured-schema reductions, anonymized contracts, synthetic parser stress cases, tool-definition bundles, secret-free provider fixture packs with request/response/tool/stop/streaming/error/limit coverage and credential-pattern rejection, and real CLI replay of every included structured-schema config;
- future corpus tests for popular model and framework configurations pinned by immutable upstream revisions.

Differential tests are especially valuable for version drift. A tokenizer or provider SDK update can change behavior without changing application code. PromptABI should make that drift visible as an artifact contract change rather than a mysterious downstream parsing failure. The deployed stop differential harness is the first provider-fixture replay line: it is intentionally small, but it already distinguishes runtime families that remove a stop delimiter from those whose generated sequence still contains it.

16 Fixture Corpus

The fixture corpus is the bridge between research claims and developer trust. It should cover popular instruct-model templates, structured-output schemas, tool-call envelopes, provider fixtures, RAG budget cases, and known delimiter collision patterns. Each entry should include provenance, license notes, exact upstream revisions, hashes, minimized repro inputs, expected diagnostics, and unsupported-fragment notes.

The current repository now contains a curated tokenizer/chat-template seed corpus spanning Llama-style header tokens, Mistral instruction tags, Qwen and ChatML im-start/im-end variants, Gemma user/model turns, Phi role tokens, DeepSeek fullwidth control tokens, Zephyr role lines, OpenAI-compatible adapter envelopes, and Alpaca-like fine-tune delimiters. The entries are intentionally minimized: they retain the structural boundary tokens and template shape needed for static analysis while excluding model weights and heavyweight tokenizer vocabularies. Each entry records roles, sentinels, expected behaviors, license notes, fixture revision, upstream reference, reproducibility notes, and the fact that no download is required.

It also contains the first real-world bug corpus. Public reports from llama.cpp, vLLM, and Hugging Face Transformers are reduced to synthetic fixtures that preserve only the interface contract failure: Phi-style user content that can render role delimiters as structure, ChatML special tokens preserved by `apply_chat_template` as apparent control text, Qwen-style XML tool-call delimiters inside free-form tool parameters, streaming tool parsing interrupted by stop strings inside `function.arguments`, array-of-object tool parameters whose item strings can leak partial XML-style tool calls, multi-function XML boundary confusion, Gemma-style quote sentinels that decode to raw string delimiters, and a llama.cpp tool-call close marker that can stop before the downstream parser is in a safe state. Each case records its public GitHub reference, bug class, minimized artifact, and expected PromptABI witness. The tests load the corpus and assert that the executable role-boundary and stop-overreachability checkers produce those witnesses, making the paper claim auditable without network calls or copied upstream code.

The corpus is executable through the public API. `load_seed_corpus()` validates coverage of all required families, checks that declared sentinels appear in either the template or special-token metadata, verifies `add_generation_prompt` declarations against the template, computes sha256 hashes for metadata and tokenizer configs, and materializes entries as PromptABI tokenizer and parsed chat-template artifacts. The same path is exposed through `promptabi corpus manifest`, which produces a deterministic manifest for CI and paper artifacts.

The structured-schema layer is executable as well. It includes a reduced open-source-agent ticket-routing schema, an anonymized markdown-fence response contract, a synthetic XML tool-call parser-broader stress case, and an OpenAI-style tool-definition bundle that now runs through the real tool-schema-ingestion CLI check, and a synthetic tool-serialization contract that drives provider/tool/parser disagreement diagnostics through the real CLI. `load_structured_schema_corpus()` validates required source-category coverage, schema/grammar/tool entry types, labels, expected parser-compatibility status, per-entry artifact hashes, and runnable PromptABI configs. `promptabi corpus structured-schema-manifest` emits a stable manifest for release checks and paper tables. The next corpus layer should add exact immutable upstream revisions for vendored tokenizer configs, llama.cpp GGUF metadata, provider traces, and structured-output libraries.

The corpus should avoid secrets and network requirements. Provider fixtures should be recorded request and response shapes, not live API calls. Model artifacts should be metadata and tokenizer files, not weights. This keeps CI fast, reproducible, and privacy-preserving.

17 Benchmark Lines

PromptABI now ships an executable CPU-only benchmark suite rather than a single smoke loop. The entrypoint `benchmarks/benchmark_smoke.py` delegates to a package-level harness that returns stable JSON records with benchmark name, iteration count, runtime, throughput, and case-specific metrics. Every case runs against real PromptABI code and checked-in examples or fixtures; no network, model weights, or provider credentials are required.

Benchmark	**Measurement**
Tokenizer analysis	Byte-level encode/decode metadata extraction across JSON-like payloads, Unicode normalization, added tokens, special tokens, and exact round-trip counts.
Template symbolic execution	Ten seed-corpus Hugging Face `tokenizer_config.json` templates parsed and boundedly symbolically executed with path and abstention counts.
Grammar emptiness	A real structured-schema fixture compiled to the bounded JSON Schema DFA product and checked against the byte-level tokenizer with state, accept, and candidate counts.
Stop checks	Stop-tokenizer alignment plus bounded stop-overreachability over structural JSON, markdown, XML-like tool-call, and provider-envelope regions.
Z3 static contracts	Cross-artifact finite obligations over prompt budgets, stop/control-token exclusion, and related constraints solved through the existing SMT layer or finite fallback.
Budget checks	Segment accounting, tokenizer-derived content counts, vLLM-style reservations, truncation simulation, required-token overflow, and finding counts.
Corpus verification	Six runnable example/fixture configs verified through the public session path with diagnostic and failing-config counts.
Cold/warm cache	Direct measurement of token-budget analysis before and after per-session cache reuse, including an explicit cache-reused metric.

The focused benchmark tests execute selected cases against repository fixtures, assert that the registry covers every deployed surface, validate the JSON schema of the benchmark output, reject unknown benchmark names, and run the CLI wrapper. A one-iteration full sweep currently reports, among other values, ten templates and 163 symbolic paths for the template case, 58 grammar states for the structured-schema grammar product, five stop-overreachability findings for the stop case, two SMT findings for the static-contract case, 32 diagnostics across six corpus configs, and a true cache-reuse observation for the warm path. These numbers are not final performance claims; they are reproducible benchlines that will make future optimization, regression detection, and paper artifact evaluation concrete.

18 Documentation and Developer Experience

A 1000-star verification repo needs more than algorithms. It needs a first-run experience where a developer immediately understands what failed and how to fix it. The README now leads with a concrete CLI command, a CPU-only explanation, and the three high-value check families. The docs tree explains architecture, quickstart, check families, the artifact model, and a focused role-boundary page that separates structural non-forgeability from semantic prompt-injection resistance while linking to the runnable unsafe/sanitized example. The contribution guide states the central norms: deterministic output, CPU-only tests, minimized fixtures, typed public APIs, and explicit soundness boundaries.

The examples directory is equally important. It now contains the minimal typed-artifact workflow, a role-delimiter injection example with failing and fixed bundles, token-budget overflow fixtures, and a RAG chunking example that exposes citation loss, boundary drift, overlap accounting, metadata inflation, template overhead, and retrieval payload truncation. PromptABI should continue accumulating small applications that intentionally contain bugs: stop overreach in JSON, provider migration that changes a tool-call AST, and tokenizer drift that changes special-token behavior. Each example should include both the failing artifact bundle and the fixed version. That gives users practical recipes and gives maintainers stable regression tests.

The CLI should remain terse by default and expandable on demand. A CI run needs deterministic diagnostics and exit codes. A local debugging session needs witnesses, rendered excerpts, token tables, exact boundary locations, and fix suggestions. A future `promptabi explain` command can bridge those modes.

19 Evaluation Design

The evaluation should answer four questions.

1. Does PromptABI find real interface bugs in popular LLM application stacks?
2. Are the findings precise enough that developers would act on them?
3. Does the verifier abstain honestly when artifacts exceed the supported fragment?
4. Do automata and SMT-backed finite contracts produce reproducible witnesses?
5. Is the runtime low enough for pull-request CI?

The experimental setup should include labeled corpora, differential agreement tests, mutation-based stress cases, and version-drift tests. Precision and recall can be measured on synthetic and real-bug corpora where ground truth is known. Differential agreement can be measured against tokenizer libraries, chat template rendering, schema validators, grammar backends, and provider fixture replay. Runtime can be reported for individual checks and corpus-wide runs. Witness quality can be evaluated by minimization size, reproducibility, and whether the witness includes enough artifact context to file an upstream bug. The deployed seed corpus already supplies the first corpus-wide experimental unit: a run can validate ten families, materialize twenty PromptABI artifacts, and compare the resulting manifest hash across revisions. Future experiments can swap minimized fixtures for revision-pinned upstream tokenizer configs while keeping the same manifest schema.

The structured-schema corpus supplies the first labeled schema/tool-call compatibility unit: tests replay expected agreement or mismatch labels through real schema and regex artifacts, execute every fixture's PromptABI config through the CLI, and produce a deterministic manifest containing source categories, expected rule IDs, hashes, and provenance.

The real-world bug corpus supplies the first labeled detection experiment. In the current snapshot, eight public bug patterns map to executable checks: Phi-style role delimiter forgery from llama.cpp, special-token injection through Hugging Face `apply_chat_template`, Qwen XML tool-call delimiter overreach from vLLM, streaming stop interruption inside OpenAI-compatible `function.arguments`, llama.cpp array-of-object tool-argument leakage, vLLM multi-function XML boundary confusion, Gemma quote-sentinel truncation from vLLM, and a llama.cpp tool-call parser-boundary stop failure. PromptABI detects all eight with bounded structural witnesses. The result is intentionally modest but important: the evaluation is no longer only synthetic, and each claimed detection is backed by a minimized fixture that can run in ordinary CPU-only CI.

The grammar-emptiness benchline adds a complementary satisfiability experiment: one fixture proves that an ordinary byte-level tokenizer can produce the bounded JSON Schema object witness `{}`, while another proves that tokenizer normalization can make a const-schema product empty. Both cases run through the same public API and CLI diagnostic contract rather than private test helpers.

The strongest paper result would be upstreamable bugs: minimized reports that maintainers of templates, adapters, structured-output libraries, or framework integrations accept as real. That evidence is more compelling than a large synthetic benchmark alone.

20 Threat Model and Non-Goals

PromptABI's threat model is structural. Attacker-controlled content may enter user messages, tool outputs, retrieval chunks, metadata, or provider-compatible fields. The verifier asks whether that content can be represented as control structure, whether output can be truncated in a way that changes parser semantics, whether a requested structured-output language is empty or ambiguous, and whether required prompt segments survive budget policies.

The non-goals are equally important. PromptABI does not prove that a model will resist prompt injection. It does not prove that a model will choose a safe tool. It does not evaluate factuality, alignment, toxicity, or behavioral compliance. It does not require model weights or generation. If an artifact contains arbitrary code, unsupported Jinja, provider behavior not captured by fixtures, or schemas outside the supported fragment, the correct behavior is to abstain or emit a bounded finding with explicit assumptions.

This narrowness makes the tool easier to trust. A structural counterexample does not depend on random seeds, temperature, model provider, or GPU hardware. If the artifact composition admits the unsafe state, that state exists.

21 Related Systems

PromptABI is adjacent to but distinct from model execution verifiers, prompt-injection scanners, schema validators, grammar-constrained decoding libraries, and provider SDK test suites. Model execution verifiers focus on tensors, devices, dtypes, shapes, checkpoint compatibility, or export gates. Prompt-injection scanners often attempt behavioral or semantic classification. Schema validators check a schema against an instance, but not necessarily the product of schema, tokenizer, grammar backend, stop policy, and application parser. Constrained-decoding libraries enforce output languages but may not report whether their language matches the downstream parser or whether stop logic truncates inside that language.

PromptABI's niche is the composition boundary. It verifies that tokenizer, template, tool, grammar, provider, and budget artifacts agree. It should integrate with existing libraries rather than replace them: validators provide ground truth for schema fragments, tokenizer libraries provide differential oracles, and provider fixtures define offline request/response semantics.

The closest analogy is a static analyzer for ABI compatibility. The concern is not whether either side of an interface is individually well-written; it is whether their composed contract remains valid as artifacts evolve.

22 Roadmap Without Step Enumeration

The roadmap is best understood as a progression from stable surfaces to formal checks to ecosystem integration. The skeleton, core artifact model, loader layer, diagnostic contract, CLI, tokenizer abstraction, finite automata layer, finite-state transducer layer, finite-contract solver, source-span tracking, differential tokenizer and stop-policy harnesses, the curated seed corpus, the manifest pipeline, and the first sanitizer-aware role-boundary checker have begun the experimental layer. The next phase should deepen stop-reachability, grammar-emptiness, and budget-survival checkers. Corpus growth, lockfiles, drift detection, SARIF, GitHub Actions, pre-commit hooks, HTML reports, suppression policies, and plugin APIs turn the verifier from a library into an everyday CI tool.

The research roadmap adds reproducibility packages, executable specifications, proof sketches for supported fragments, mutation-based fuzzing, cross-version CI, and real-bug benchmark suites. The community roadmap adds contribution infrastructure, compatibility matrices, integration guides, launch assets, and governance. These activities are not separate from the technical work. They are how a formal verifier becomes durable enough for production teams to trust and for researchers to reproduce.

23 Current Smoke Results

The current codebase was validated with focused tests for the package, artifact model, formal core, tokenizer abstraction, source-span mapping, configuration loader, public API, and CLI workflow. The tokenizer tests exercise normalization, added tokens, special tokens, round-trip metadata, Hugging Face ByteLevel BPE behavior, `tiktoken cl100k_base`, and a trained SentencePiece model. The CLI tests exercise config discovery from nested directories, artifact-location overrides, deterministic JSON output, SARIF output, missing-artifact failures, cache directory preparation, quiet/verbose text controls, and configurable exit-code thresholds. Structured snapshot tests now lock representative error, warning, and info diagnostics across text, JSON, and SARIF, including artifacts, spans, witnesses, suggestions, fingerprints, and check modes. The public API and CLI tests also prove that check-mode metadata is preserved across diagnostic objects, JSON, human text, and SARIF. New source-span tests prove that PromptABI config declarations, tokenizer `chat_template` fields, JSON Schema fields, tool definition names, and malformed JSON parse errors produce exact line-oriented spans against real files. The formal tests exercise automata intersection, difference, emptiness, shortest witnesses, transducer composition, projection, over-approximation, finite-domain counterexample extraction, minimal unsat cores, and the installed Z3 backend. The new corpus tests validate required family coverage, sentinel/template consistency, metadata hashes, reproducibility metadata, artifact materialization through the real offline loader, manifest generation, manifest writing, and the `promptabi corpus manifest` CLI. The structured-schema corpus tests validate required source categories, schema/grammar/tool-definition artifact materialization, tool-schema ingestion, labeled parser-compatibility replay, tool-serialization fixture replay, per-fixture CLI diagnostics, manifest writing, and the `promptabi corpus structured-schema-manifest` CLI. Stop differential tests replay recorded provider-style chunks through CPU-only OpenAI-compatible and Hugging Face simulators, assert delimiter-retention behavior, exercise mismatch diagnostics through the real CLI, and the `examples/stop-policies` workflow emits a concrete `stop-differential-agreement` diagnostic for a vLLM-compatible fixture. The real-world bug corpus tests load eight public GitHub bug reductions and prove that the current role-boundary and stop-overreachability implementations emit the expected minimized witnesses against executable code, including the new array-of-object tool-parameter witness path. The CLI was also run against the minimal typed-artifact example configuration, producing a passing informational diagnostic. The smoke benchmark runs the skeleton verifier repeatedly on the example config and reports a deterministic JSON result. These are small but important baselines: they prove that the repository can already execute code, not just describe a future tool.

Line	Current status
Focused tests	Artifact model, public API, config loading, and CLI workflow behavior pass in isolation.
Formal core tests	Automata reachability and difference, transducer composition/projection/over-approximation, finite-domain solving, unsat-core extraction, and Z3-backed assignment extraction pass against executable code.
Check-mode taxonomy tests	Diagnostic objects, JSON output, text output, and SARIF output preserve sound/complete/bounded/Z3-backed/heuristic/abstaining metadata deterministically.
Tokenizer differential tests	Byte-level normalization and added-token behavior, Hugging Face ByteLevel BPE, `tiktoken`, and SentencePiece adapters match real backend behavior.
Chat-template symbolic tests	Hugging Face `tokenizer_config.json` templates across ten seed families expose template source, special tokens, generation prompts, roles, fields, tools, filters, whitespace controls, bounded symbolic paths, and unsupported constructs.
Role-boundary checker tests	Bounded non-forgeability diagnostics catch dynamic role-header forgery and user-content delimiter forgery across ChatML, Llama, Mistral, XML tool tags, markdown fences, and fine-tune hash headers; witnesses include minimized inputs, rendered excerpts, centered token evidence, and exact offsets, while sanitizer tests prove JSON/escape filters plus sanitizer-preserving bindings suppress only protected-expression false positives.
Real-world bug corpus	Eight public llama.cpp/vLLM/Hugging Face bug patterns are reduced to synthetic offline fixtures and detected by the role-boundary and stop-overreachability checkers with concrete witnesses.
Stop differential tests	Recorded provider/framework chunks replay through CPU-only stop simulators; OpenAI-compatible/vLLM traces exclude delimiters, Hugging Face traces retain them, and CLI diagnostics flag mismatches with witness steps.
CLI smoke	`promptabi verify -config examples/minimal/promptabi.json` returns pass, and `-fail-on`, `-artifact`, `-cache-dir`, `-quiet`, and `-verbose` are covered.
Corpus smoke	`promptabi corpus manifest` validates ten minimized instruct-template families and emits stable per-entry hashes without downloads.
Structured-schema corpus	`promptabi corpus structured-schema-manifest` validates labeled structured-output/tool-calling fixtures from OSS reductions, anonymized contracts, and synthetic stress cases; tests replay parser compatibility, tool-schema ingestion, tool serialization, and CLI configs.
Token-budget and RAG modeling	`promptabi verify -config examples/token-budget/promptabi.json` models named must-survive segments, framework context reservations, config/artifact context-window precedence, required-token overflow, framework truncation decisions, bounded survival proofs, and minimized counterexamples; `examples/rag-chunking/promptabi.json` adds deterministic chunking diagnostics for tokenizer mismatch, boundary drift, citation loss, overlap errors, metadata inflation, prompt-template overhead, and retrieval payload truncation.
Tokenizer/config drift	Focused analyzer and CLI tests compare baseline/current tokenizer directories and flag special-token ID, added-token, normalizer, chat-template, BOS/EOS, and stop-

These are not the final experiments. They are the baseline benchlines that later experiments will extend: every new checker should add focused tests, fixture cases, corpus cases where relevant, and a benchmark line that can be compared across revisions.

24 Why CPU-Only Is a Strength

Many LLM quality questions are inherently behavioral and require model access. PromptABI intentionally avoids those claims. Tokenization runs on CPU. Chat-template rendering runs on CPU. JSON Schema normalization, grammar compilation, automata and transducer operations, parser checks, source-span tracking, fixture replay, and truncation simulation run on CPU. Provider compatibility can be tested from recorded fixtures. Context-window analysis depends on token counts and framework policies, not logits.

This makes PromptABI deployable in environments where GPU inference, external API calls, and prompt upload are not acceptable. A company can run the verifier inside CI on private prompt artifacts. A maintainer can run corpus checks on a laptop. A paper artifact can be reproduced without model weights. A pull request can fail because a tokenizer config changed, not because a stochastic generation happened to expose the bug once.

The CPU-only boundary also sharpens the formal claims. The tool can say "this delimiter is forgeable" or "this stop sequence can truncate valid JSON" with a structural witness. It should not say "the model is safe." That honesty is a competitive advantage.

25 Conclusion

PromptABI is positioned to verify a layer of LLM systems that is both fragile and under-tooled: the discrete interface between application data structures, prompt rendering, tokenization, constrained output, provider protocols, and application parsing. The first repository milestone establishes the engineering foundation: typed package, artifact model, tokenizer abstraction, CLI, public API, diagnostics, tests, docs, examples, fixtures, benchmarks, and contribution guidance. The research foundation is the transducer view of prompts, tokenizers, grammars, stop policies, parsers, and budget policies.

The next milestones should add the first formal checkers without changing the public surface. If the project maintains deterministic outputs, minimized witnesses, clear abstention boundaries, and real corpus validation, it can become both a practical CI tool and a credible systems research contribution. The core insight is simple but powerful: many production LLM failures happen before the neural network is involved, and those failures can be found before deployment.